

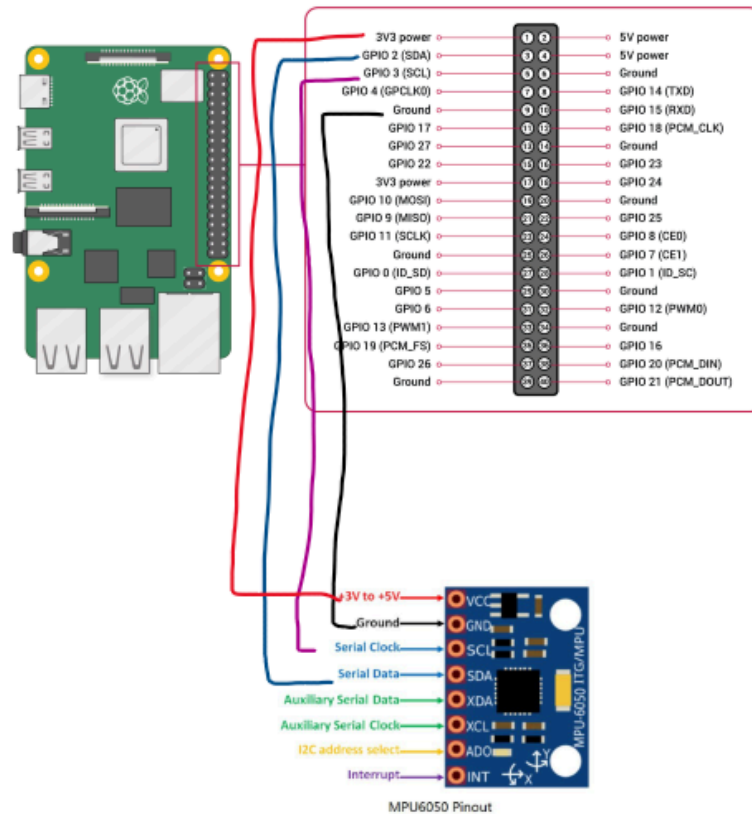
Lab 4: Inertial Measurement Units

Katelyn Nguyen
Oluwasegun Oluwole
CSCE 462 - 500
Group 1

27 September 2024

Hardware Description

Below is the diagram for the hardware used in building this system. The VCC pin of the MPU IMU 6050 chip is connected to the 3v3 pin of the Raspberry Pi 3B+ via a jumper wire. The SDA and SCL chips are connected to their respective pins on the Raspberry Pi via jumper wires; SDA is used for the transmission of data and SCL is used to synchronize the communication between the master and slave device. Finally, the GND pin on the IMU chip is connected to the ground pin on the raspberry pi.



Software Description

Below is the beginning of the code where necessary libraries and imports are performed as well as the initialization of the i2c protocol and the mpu.

```

import board
import busio
import adafruit_mpu6050
import RPi.GPIO as GPIO

#perf_counter is more precise than time() for dt calculation
from time import sleep, perf_counter
import math
import numpy as np
import matplotlib.pyplot as plt

i2c = busio.I2C(board.SCL, board.SDA)
mpu = adafruit_mpu6050.MPU6050(i2c)

```

A try-finally block is then entered into. In the beginning, we enter into a calibration phase where the person that the mpu is attached to, stands still. The x,y, and z acceleration values recorded here are used to calculate the magnitude of the acceleration to be used as a baseline to compare to when the person is walking. There is a sleep after the Begin Stepping print statement so the human has time to read the print statement.

```

try:

    # BEGIN THE "CALIBRATION PHASE"

    print("Calibrating...")

    #read one data point while staying still to act as our calibration values
    x_acc, y_acc, z_acc = mpu.acceleration

    #calculate the magnitude, save as the control/calibration value
    calib_mag = math.sqrt(x_acc**2+y_acc**2+z_acc**2)

    print("\n*****\nBegin Stepping...\n*****")
    sleep(1)

```

Below in the try-finally block is the movement phase where step count is initialized to 0 and start time is set to perf_counter(). Then a while loop is entered until 60 seconds have elapsed or 50 steps have been taken. Inside the while loop, the x,y, and z acceleration values are used to calculate the magnitude of acceleration. If the magnitude is greater than or less than 10 percent of the calibrated magnitude from the calibrated phase, that means a step has been taken and step count is incremented by 1. There is a sleep of 0.4 after a step has been recorded to avoid excess steps being recorded due to noise. Finally, the GPIO pins on the raspberry pi are cleaned up.

```
47         # BEGIN THE "MOVEMENT PHASE"
48
49         step_count = 0
50         start_time = perf_counter()
51
52         #write a loop to poll each sensor and print its axis values
53         while ((perf_counter() - start_time) < 60 and step_count < 52):
54             #read a new data point (x,y,z) of acceleration from the MCP6050
55             x_acc, y_acc, z_acc = mpu.acceleration
56
57             print("")
58             sleep(0.1)
59
60             #calculate the magnitude of the acceleration
61             mag = math.sqrt(x_acc**2+y_acc**2+z_acc**2)
62
63             #if the current magnitude is outside +-10% of the calibrated magnitude, count it as a step
64             if (mag < 0.9*calib_mag or mag > 1.1*calib_mag):
65                 step_count += 1
66                 print("step count: ",step_count)
67
68                 #delay to avoid double counting a step
69                 sleep(.4)
70
71     finally:
72         GPIO.cleanup()
```

Lab Questions

Provide the **questions and answers** to the questions included in the lab manual.

1. How frequently can the sensor be polled? Is this fast enough for this lab? Why or why not?

The sensor is polled at a frequency of approximately 400kHz. This is fast enough for the lab because we were able to poll data points at a rate that kept up with our step cadence.

2. Did you apply filter(s) to sensor readings? If yes , how did you use the filter(s)? If no, why?

No we did not apply filters to the sensor readings because we just calculated the magnitude of acceleration and compared it to value that was calculated when there was no movement.

3. Explain your step counting algorithm and its accuracy.

In the calibration phase, a single data point is collected from the accelerometer when the device is at rest, ensuring no movement. This reading is used to calculate the magnitude of the acceleration vector, which serves as a baseline reference for detecting movement. As the movement phase begins, a step counter is initialized to 0, and the start time is recorded. A `while` loop then runs for a maximum of 52 steps or 60 seconds, whichever comes first. Inside the loop, the x, y, and z acceleration values are continuously collected and used to calculate the magnitude of acceleration. If the magnitude exceeds or falls below 10% of the calibrated magnitude (collected during the calibration phase), a step is counted, and a delay of 0.4 seconds is introduced to account for the natural rhythm of walking and prevent multiple counts for a single step. The algorithm was accurate when a natural walking rhythm was undertaken, but when steps that were too heavy caused micro up and down movement in the body, the algorithm would count multiple steps. In the lab demo, the algorithm counted 30 steps and there were actually 33 steps that were walked (e.g. 91% accuracy).